

Kolmogorov-Arnold Networks for Online Reinforcement Learning

Victor A. Kich^{1*}, Jair A. Bottega^{1*}, Raul Steinmetz², Ricardo B. Grando³, Ayano Yorozu¹ and Akihisa Ohya¹

¹Intelligent Robot Laboratory, University of Tsukuba,
Tsukuba, Japan (victorkich98@gmail.com, jairaugustobottega@gmail.com) * Corresponding author

²Technology Center, Federal University of Santa Maria,
Santa Maria, Brazil (rsteinmetz@inf.ufsm.br)

³Robotics and AI Lab, Technological University of Uruguay,
Rivera, Uruguay (ricardo.bedin@utec.edu.uy)

Abstract: Kolmogorov-Arnold Networks (KANs) have shown potential as an alternative to Multi-Layer Perceptrons (MLPs) in neural networks, providing universal function approximation with fewer parameters and reduced memory usage. In this paper, we explore the use of KANs as function approximators within the Proximal Policy Optimization (PPO) algorithm. We evaluate this approach by comparing its performance to the original MLP-based PPO using the DeepMind Control Proprio Robotics benchmark. Our results indicate that the KAN-based reinforcement learning algorithm can achieve comparable performance to its MLP-based counterpart, often with fewer parameters. These findings suggest that KANs may offer a more efficient option for reinforcement learning models. Our implementations can be found in the following link: <https://github.com/victorkich/Kolmogorov-PPO>.

Keywords: Deep Reinforcement Learning, Robotics, Kolmogorov-Arnold Networks

1. INTRODUCTION

Kolmogorov-Arnold Networks (KANs) [1] are learnable approximators based on the Kolmogorov-Arnold representation theorem [2], which states that any multivariate continuous function can be expressed as a finite sum of continuous univariate functions. KANs can approximate functions with fewer neurons compared to Multi-Layer Perceptrons (MLPs), thereby reducing network complexity. This simplification enables easier function decomposition, allowing for more straightforward and targeted architecture designs tailored to specific problems. Additionally, KANs utilize less memory and enhance model interpretability, making them highly appealing for various machine learning applications.

Reinforcement Learning (RL)[3] has proven efficient for continuous control tasks such as autonomous robot manipulation[4], navigation [5], and video game playing [6], [7]. Proximal Policy Optimization (PPO)[8] is a Deep RL algorithm that employs model-free online reinforcement learning to approximate an optimal policy over online training experiences. This algorithm is a cornerstone in the field, excelling in numerous benchmarks and serving as the foundation for newer models like Multi-Agent PPO [9], PPO with Covariance Matrix Adaptation [10], PPO with Action Mask [11], and PPO with Policy Feedback [12].

Networks with fewer parameters yet equal or superior approximation capabilities have the potential to significantly enhance performance in various deep reinforcement learning applications. In this work, we propose the use of KANs as policy and value approximators for PPO. The main contributions of this paper are:

- The first application of KANs as function approximators in a reinforcement learning algorithm.
- A performance comparison between KAN-based and MLP-based Proximal Policy Optimization in robotics con-

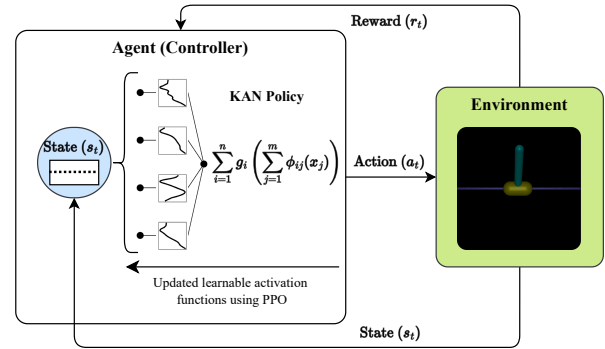


Fig. 1.: Overview of the proposed framework.

trol.

The paper is organized as follows: In Section 2, we review related works and research in the field. Section 3. provides the mathematical background of the techniques employed. The proposed methodology is presented in Section 4. Section 5.presents the results and evaluation analysis. Finally, we discuss our findings in Section 6.

2. RELATED WORKS

KANs have recently garnered significant attention for their versatility and enhanced performance across various applications. Numerous studies have explored KANs as alternatives to traditional Multilayer Perceptrons, consistently demonstrating their superiority in specific scenarios. Concurrently, online reinforcement learning has been a cornerstone in the domain of continuous control tasks, underscoring the importance of investigating KANs' potential in this field. Evaluating whether KANs can compete with or surpass MLPs in reinforcement learning applications is a critical area of exploration. This section reviews

pivotal research contributions relevant to this research.

Expanding on the KAN framework, Bozorgasl and Chen [13] introduced Wav-KAN, a neural network architecture that integrates wavelet functions [14] into the KAN framework. This innovation addresses common challenges faced by traditional MLPs, such as interpretability, training speed, robustness, computational efficiency, and performance. By incorporating wavelet functions, Wav-KAN effectively captures both high-frequency and low-frequency components of the input data, enhancing overall performance.

Exploring KANs' performance on predicting time series, Rubio et al. [15] demonstrated the effectiveness of these networks in predicting satellite traffic, showing their superiority over traditional MLPs for time series analysis. Similarly, Genet and Hugo [16] introduced Temporal Kolmogorov-Arnold Networks (TKANs), combining elements of KANs and Long Short-Term Memory (LSTM) networks to improve time series forecasting.

Expanding on the exploration of KANs' capabilities, Yang, Qin, and Yu [17] proposed a novel framework that enhances the interpretability of neural cognitive diagnosis models (CDMs) using KANs while maintaining high accuracy. They introduced two methods: replacing traditional MLPs in existing neural CDMs with KANs and designing a new aggregation framework entirely based on KANs. Their experimental results on four real-world datasets showed that the KA2NCD framework outperforms traditional and neural CDMs in both accuracy and interpretability, demonstrating its potential in intelligent education systems.

Online Reinforcement Learning, where an agent continuously learns from ongoing interaction with the environment, instead of using memory replay, has been successfully applied to various robotic tasks. In the humanoid robots area, Melo et al. [18] successfully utilized PPO to develop running skills in a humanoid robot, using the RoboCup 3D Soccer Simulation environment, where humanoid robots compete in simulated soccer matches. Expanding the use of online reinforcement learning to other areas, Proctor [19] used the algorithm for localizing and searching for lost nuclear sources in a designated area, which is vital for societal safety and reducing human harm.

To the best of our knowledge, no efforts have been made to apply KANs as alternatives to MLPs in online reinforcement learning, making this study a pivotal contribution.

3. THEORETICAL BACKGROUND

This section presents the theory and mathematical foundations underpinning our approach.

3.1 Kolmogorov-Arnold Networks

Kolmogorov-Arnold Networks leverage the Kolmogorov-Arnold representation theorem, which asserts that any multivariate continuous function can be decomposed into a

finite composition of univariate functions and addition operations. Formally, for a smooth function $f : [0, 1]^n \rightarrow \mathbb{R}$, this can be expressed as:

$$f(x) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \varphi_{q,p}(x_p) \right), \quad (1)$$

where $\varphi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ and $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$ are continuous functions.

In KANs, weight parameters are replaced by learnable 1D functions φ , parametrized as B-splines. The computation in a KAN layer with n_{in} inputs and n_{out} outputs is:

$$x_{l+1,j} = \sum_{i=1}^{n_l} \varphi_{l,j,i}(x_{l,i}), \quad (2)$$

where $\varphi_{l,j,i}$ is a spline function connecting the i -th neuron in layer l to the j -th neuron in layer $l + 1$.

The backpropagation process in KANs involves calculating gradients of the spline functions. The loss $\mathcal{L}$ is minimized using gradient descent, with the gradient of the loss with respect to the spline parameters c_i computed as:

$$\frac{\partial \mathcal{L}}{\partial c_i} = \sum_{j=1}^{n_{\text{out}}} \frac{\partial \mathcal{L}}{\partial x_{l+1,j}} \frac{\partial x_{l+1,j}}{\partial c_i}, \quad (3)$$

where $\frac{\partial x_{l+1,j}}{\partial c_i}$ involves the derivative of the spline function with respect to its coefficients.

3.1.1 Grid and Order in KANs

In KANs, the spline functions φ are defined over a discretized domain, commonly referred to as a *grid*. The **grid** determines the points at which the spline functions are evaluated, essentially specifying the resolution of the function approximation. The **order** parameter controls the degree of the B-splines used to represent the functions. For example, an order of 1 corresponds to linear splines, while higher orders correspond to splines with more complex shapes, allowing for more flexible function approximations.

The combination of the *grid* and *order* parameters defines the complexity and capacity of the KANs to model intricate functional relationships. A finer grid with higher-order splines enables more accurate approximations of complex functions, at the cost of increased computational complexity.

3.1.2 Comparison of KANs and MLPs

- **Learnable Activation Functions:** KANs feature learnable activation functions on edges, while MLPs have fixed activation functions on nodes.
- **Spline-Based Weights:** KANs use splines to represent weights, enhancing their capacity to approximate complex functions with fewer parameters.
- **Layer Extension:** Both KANs and MLPs can be extended to multiple layers.

3.2 Proximal Policy Optimization

Proximal Policy Optimization is a policy gradient method that computes an estimator of the policy gradient for use in a stochastic gradient ascent algorithm. The policy gradient estimator is:

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right], \quad (4)$$

where π_{θ} is the stochastic policy, and $\hat{A}_t$ is an estimator of the advantage function at timestep t . The expectation $\hat{\mathbb{E}}_t[\cdot]$ denotes the empirical average over a finite batch of samples.

This estimator is derived from the objective function:

$$L_{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]. \quad (5)$$

To prevent excessively large policy updates, PPO introduces a clipped surrogate objective. Let $r_t(\theta)$ denote the probability ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (6)$$

The clipped surrogate objective is defined as:

$$L_{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (7)$$

where ϵ is a hyperparameter, typically set to 0.2. This objective encourages the new policy to stay within a small region around the old policy by clipping the probability ratio $r_t(\theta)$ when it exceeds the interval $[1 - \epsilon, 1 + \epsilon]$.

The advantage function $\hat{A}_t$ can be estimated using Generalized Advantage Estimation (GAE), which reduces variance while introducing a bias controlled by parameter λ :

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad (8)$$

where δ_t is the temporal difference (TD) error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \quad (9)$$

In practice, PPO uses a combination of the clipped surrogate objective, a value function loss, and an entropy bonus to ensure sufficient exploration. The combined objective function is:

$$L_{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t [L_{CLIP}(\theta) - c_1 L_{VF}(\theta) + c_2 S[\pi_{\theta}](s_t)], \quad (10)$$

Algorithm 1 Proximal Policy Optimization

```

1: Initialize policy parameters  $\theta$ 
2: for each iteration do
3:   for actor = 1, 2, ..., N do
4:     Run policy  $\pi_{\theta}$  in environment for  $T$  timesteps
5:     Compute advantage estimates  $\hat{A}_t$ 
6:   end for
7:   Optimize surrogate  $L_{CLIP}(\theta)$  with  $K$  epochs and minibatch size  $M \leq NT$ 
8:   Update policy parameters  $\theta$ 
9: end for

```

where $L_{VF}(\theta) = (V_{\theta}(s_t) - V_t^{\text{target}})^2$ is the squared-error loss for the value function, $S[\pi_{\theta}](s_t)$ is the entropy bonus, and c_1, c_2 are coefficients.

The PPO technique (Algorithm 1) alternates between collecting data through interaction with the environment and optimizing the policy using the clipped surrogate objective.

4. METHODOLOGY

We conducted our experiments using six environments from the Gymnasium Mujoco suite, specifically adapted from the DeepMind Control Proprio benchmarks [20]. These environments are well-suited for evaluating continuous control tasks and include: HalfCheetah-v4, Hopper-v4, InvertedPendulum-v4, Pusher-v4, Swimmer-v4, and Walker2d-v4. These environments provide a diverse set of challenges, testing the ability of the agent to learn different types of motion and control strategies.

Our implementation utilizes the PPO algorithm as the core reinforcement learning method. PPO is a state-of-the-art algorithm known for its stability and efficiency in policy gradient methods [8]. We incorporated KANs as the function approximators for both the policy (actor) and value (critic) functions, replacing the traditional MLP used in standard PPO implementations.

4.1 Network Architectures

We designed and compared several network architectures for the policy (actor) and value (critic) functions to evaluate the effectiveness of KANs versus MLPs. The configurations included:

- **MLP (a=2, c=2)**: Two hidden layers for both actor and critic with 64 units each.
- **MLP (a=1, c=2)**: One hidden layer for the actor and two hidden layers for the critic, with 64 units each.
- **KAN (k=2, g=3)**: KAN for the actor with no hidden layers, order $k = 2$ and grid $g = 3$, and a critic implemented as an MLP with 2 hidden layers of 64 units each.
- **Full KAN (k=2, g=3)**: Both the actor and critic networks implemented using KANs with no hidden layers, order $k = 2$ and grid $g = 3$.

The parameter settings for KANs were chosen based on preliminary tests where we evaluated various configurations to identify the most effective setup. Our tests demonstrated that a KAN with no hidden layers, order $k = 2$, and grid $g = 3$ provided the best performance in terms of computational efficiency and approximation capability. Consequently, we adopted this architecture for intensive testing.

For the Kolmogorov-Arnold Network, we set the parameters $k = 2$ and $g = 3$ based on these preliminary findings, ensuring the network's ability to approximate the continuous functions involved in control tasks while maintaining low computational complexity. The number of parameters in the KAN configurations is significantly lower compared to the MLP counterparts, highlighting the efficiency of KANs. The average number of parameters for each configuration across the environments is shown

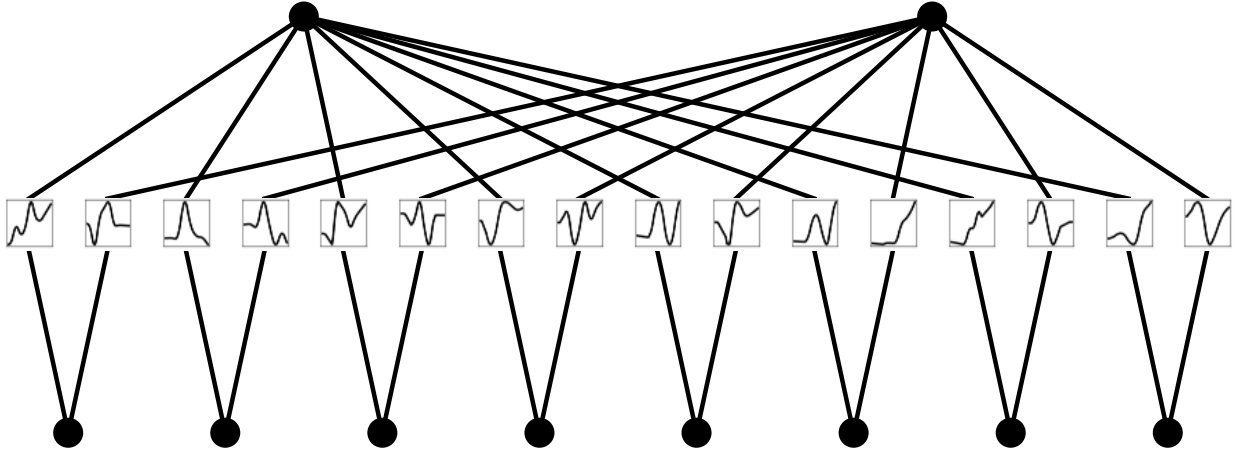


Fig. 2.: KAN architecture of the environment DMC Proprio Swimmer-v4.

in Table 1. An Kolmogorov-Arnold Network example is depicted in Fig.2.

4.2 Training Procedure

Each agent was trained for 1 million steps in each environment. We used the following hyperparameters for training:

- **Number of Environments:** 1
- **Learning Rate:** 3×10^{-4}
- **Clip Parameter** ϵ : 0.2
- **Number of Epochs:** 10
- **Batch Size:** 64
- **Discount Factor** γ : 0.99
- **GAE Parameter** λ : 0.95
- **Seeds:** [0, 1, 2, 3, 4]
- **Device:** CPU

The agents' performance was evaluated based on the average cumulative reward obtained over 100 evaluation episodes after the training period. The evaluation episodes were conducted without exploration noise to accurately assess the learned policies' effectiveness.

5. RESULTS

The performance of the KAN-based PPO was evaluated against the MLP-based PPO using metrics such as average return and the number of parameters. The average return, defined as the mean cumulative reward over 100 evalua-

Table 1.: Number of parameters (actor) for different DMC Proprio environments.

Task	MLP	MLP	KAN
Settings	(a=2, c=2)	(a=1, c=2)	(k=2, g=3)
HalfCheetah-v4	5702	1542	510
Walker2d-v4	5702	1542	510
Hopper-v4	5123	963	165
InvertedPendulum-v4	4545	385	20
Swimmer-v4	4866	706	80
Pusher-v4	6151	1991	805
Average of Parameters	5348	1188	348

tion episodes, provided a robust measure of the policy's effectiveness. The number of parameters, representing the total count of trainable elements in the network, offered insights into the efficiency and potential computational savings achieved by using KANs.

5.1 Performance Metrics

Figure 3 illustrates the average rewards obtained across the training steps for each environment. In general, the KAN-based PPO matched or exceeded the performance of the MLP-based PPO in most environments. This demonstrates the potential of KANs to achieve robust performance with fewer parameters, particularly in the task HalfCheetah-v4, where the KAN-based models outperformed their MLP counterparts.

Table 2 shows the DMC Proprio scores for the control inputs at 1 million training steps. The KAN-based models performed competitively, sometimes surpassing the MLP configurations in terms of reward mean while maintaining a lower parameter count. This highlights the potential of KANs to provide efficient and effective learning with potentially reduced computational requirements.

5.2 Discussion

The results from our experiments indicate that KANs, when integrated into the PPO framework, can achieve performance comparable to traditional MLPs with significantly fewer parameters. This demonstrates the potential efficiency of KANs in reducing the computational complexity of reinforcement learning models without compromising performance.

While KAN-based models generally matched or exceeded the performance of MLP-based models, these improvements were context-dependent. For example, in tasks such as HalfCheetah-v4 and Hopper-v4, the KAN-based PPO not only achieved higher rewards but also maintained a lower parameter count, highlighting its efficiency. However, in environments like Pusher-v4 and Walker2d-v4, the performance gains were less pronounced or even inferior, suggesting that the benefits of KANs can vary depending on task complexity. Adjustments in param-

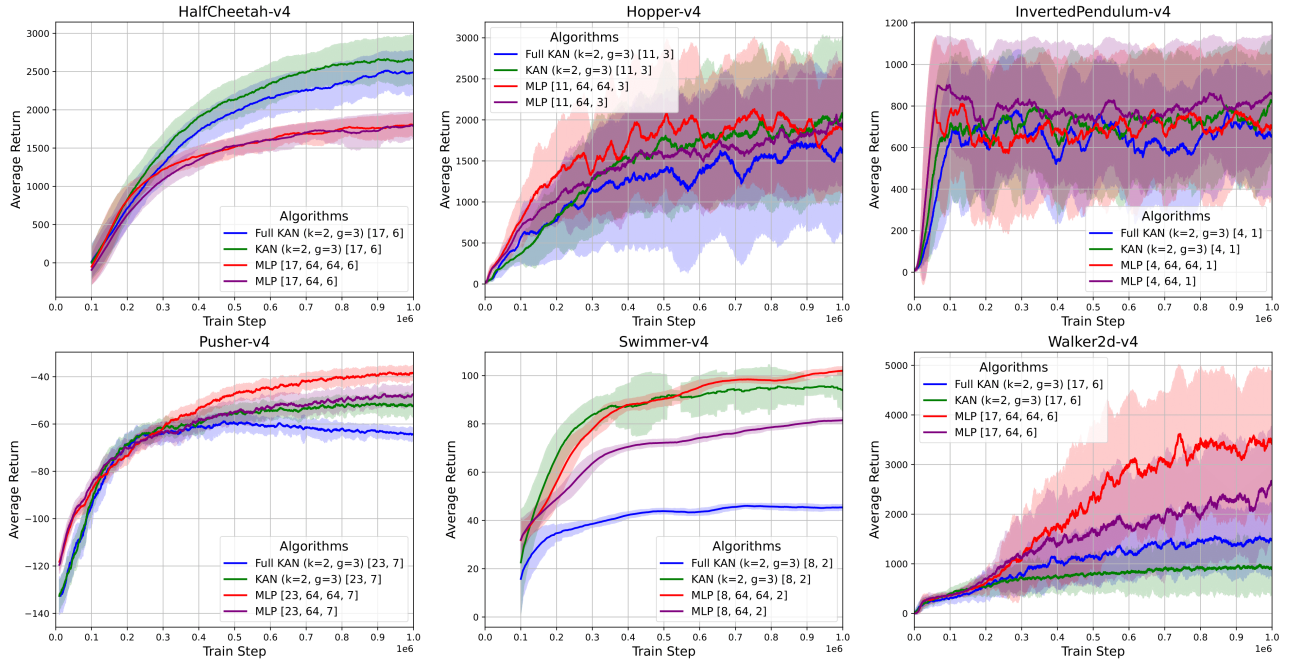


Fig. 3.: Reward average comparison for all the trained environments using the proposed architectures (using 5 seeds).

Table 2.: DMC Proprio scores for control inputs at 1M train steps. Scores between the top 5% are shown in bold format.

Task	Full KAN (k=2, g=3)	KAN (k=2, g=3)	MLP (a=2, c=2)	MLP (a=1, c=2)
<i>Train Steps</i>	1M	1M	1M	1M
HalfCheetah-v4	2490	2644	1803	1791
Hopper-v4	1633	2031	1897	1918
InvertedPendulum-v4	653	818	694	861
Pusher-v4	-64	-52	-39	-47
Swimmer-v4	45	94	102	82
Walker2d-v4	1487	889	3439	2593
<i>Reward Mean</i>	1358	1540	1482	1399
<i>Average of Parameters (actor+critic)</i>	415	5490	10490	6330

eters, grid searches, or incorporating additional neurons could potentially enhance performance in these cases.

The reduced parameter count in KANs translates to lower memory usage and potentially faster training times, making them suitable for deployment in resource-constrained environments. For instance, the KAN (k=2, g=3) architecture required significantly fewer parameters compared to the MLP (a=2, c=2) configuration, averaging 348 versus 5348 parameters, respectively. This substantial reduction underscores the efficiency of KANs in achieving comparable performance with fewer resources.

Additionally, pruning operations—where less important parameters are eliminated—can further streamline KAN models, enhancing efficiency without compromising performance. This pruning process is particularly beneficial for KANs due to their already reduced parameter count, allowing the model to focus on the most critical parameters. The inherent interpretability of KANs, based on the Kolmogorov-Arnold representation theorem, also provides advantages for understanding and debugging learned policies, which is crucial in applications where model transparency and explainability are important. However, continued exploration and optimization of KAN architec-

tures are necessary to fully leverage their potential across various reinforcement learning applications.

KANs, while providing significant benefits in terms of parameter count and interpretability, currently face challenges in computational speed. In our evaluation of 1000 steps on the HalfCheetah environment, the MLP with 2 hidden layers of 64 units completed the task significantly faster, averaging 0.37 seconds compared to the KAN (k=2, g=3), which took 3.39 seconds. This slower performance stems from the complex calculations required by the B-splines used as activation functions, which, despite reducing the number of parameters, introduce a trade-off between parameter efficiency and execution speed—especially critical in real-time applications. MLPs, though more parameter-heavy, benefit from highly optimized implementations in libraries like Torch, resulting in faster execution times. The novelty of KANs means there is a lack of specialized optimization techniques, presenting a clear need for future research to enhance their computational efficiency.

Addressing these computational bottlenecks will be essential to fully leverage the potential of KANs in reinforcement learning tasks. Optimizing spline calculations

and developing KAN-specific computational strategies could improve their competitiveness in terms of execution speed, making them more viable for applications that demand real-time decision-making while maintaining their advantages in memory usage and training efficiency. Future work should focus on overcoming these challenges to realize the full capabilities of KANs in practical settings.

6. CONCLUSION

Our study demonstrated that Kolmogorov-Arnold Networks can achieve performance comparable to traditional Multi-Layer Perceptrons with significantly fewer parameters in reinforcement learning tasks. This makes KANs a promising alternative, especially in resource-constrained environments where memory and computational efficiency are critical. However, the current limitations in computational speed due to the complexity of B-spline activation functions suggest the need for further research and optimization. Future work should focus on developing specialized optimization techniques and frameworks for KANs to improve their computational efficiency and make them more suitable for real-time applications.

ACKNOWLEDGEMENT

This work was supported by the Ministry of Education, Culture, Sports, Science and Technology (MEXT) scholarship. We are also grateful for the support provided by the Human-Centered AI program at the University of Tsukuba.

REFERENCES

- [1] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "Kan: Kolmogorov-arnold networks," *arXiv preprint arXiv:2404.19756*, 2024.
- [2] A. Kolmogoroff, "On the representation of continuous functions of several variables by superpositions of continuous functions of one variable and addition," in *Dokl. Akad. Nauk SSSR*, vol. 114, 1957, pp. 953–956.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [4] H. Nguyen and H. La, "Review of deep reinforcement learning for robot manipulation," in *2019 Third IEEE International Conference on Robotic Computing (IRC)*. IEEE, 2019, pp. 590–595.
- [5] K. Zhu and T. Zhang, "Deep reinforcement learning based mobile robot navigation: A review," *Tsinghua Science and Technology*, vol. 26, no. 5, pp. 674–691, 2021.
- [6] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse domains through world models," *arXiv preprint arXiv:2301.04104*, 2023.
- [7] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [9] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 611–24 624, 2022.
- [10] P. Hämmäläinen, A. Babadi, X. Ma, and J. Lehtinen, "Ppo-cma: Proximal policy optimization with covariance matrix adaptation," in *2020 IEEE 30th International Workshop on Machine Learning for Signal Processing (MLSP)*. IEEE, 2020, pp. 1–6.
- [11] C.-Y. Tang, C.-H. Liu, W.-K. Chen, and S. D. You, "Implementing action mask in proximal policy optimization (ppo) algorithm," *ICT Express*, vol. 6, no. 3, pp. 200–203, 2020.
- [12] Y. Gu, Y. Cheng, C. P. Chen, and X. Wang, "Proximal policy optimization with policy feedback," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 7, pp. 4600–4610, 2021.
- [13] Z. Bozorgasl and H. Chen, "Wav-kan: Wavelet kolmogorov-arnold networks," *arXiv preprint arXiv:2405.12832*, 2024.
- [14] A. Antoniadis, "Wavelets in statistics: a review," *Journal of the Italian Statistical Society*, vol. 6, pp. 97–130, 1997.
- [15] C. J. Vaca-Rubio, L. Blanco, R. Pereira, and M. Caus, "Kolmogorov-arnold networks (kans) for time series analysis," *arXiv preprint arXiv:2405.08790*, 2024.
- [16] R. Genet and H. Inzirillo, "Tkan: Temporal kolmogorov-arnold networks," *arXiv preprint arXiv:2405.07344*, 2024.
- [17] S. Yang, L. Qin, and X. Yu, "Endowing interpretability for neural cognitive diagnosis by efficient kolmogorov-arnold networks," *arXiv preprint arXiv:2405.14399*, 2024.
- [18] L. C. Melo, D. C. Melo, and M. R. Maximo, "Learning humanoid robot running motions with symmetry incentive through proximal policy optimization," *Journal of Intelligent & Robotic Systems*, vol. 102, no. 3, p. 54, 2021.
- [19] P. Proctor, C. Teuscher, A. Hecht, and M. Osiński, "Proximal policy optimization for radiation source search," *Journal of Nuclear Engineering*, vol. 2, no. 4, pp. 368–397, 2021.
- [20] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. d. L. Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq *et al.*, "Deepmind control suite," *arXiv preprint arXiv:1801.00690*, 2018.